设计模式面试题速记通关版 _ 面试刷题 mianshiya.com

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*

什么是命令模式？一般用在什么场景？

命令模式把“请求”封装成一个对象，这样就能让方法参数化。比如你调用一个方法，传进去的不是一个值，而是一个命令对象，它知道要执行什么动作、由谁来执行。

这样一来，调用方和执行方就彻底解耦了。调用方只管触发 execute()，压根不经过具体业务逻辑，后面的Receiver自己处理。常见的像JDK里的 Runnable 就是典型例子，你把任务提交给线程池，至于怎么执行，调用者不用管。

1）支持撤销和重做。命令对象可以记录执行前的状态，undo() 时恢复回去。比如文本编辑器里的 Ctrl+Z，每个操作都是一个命令。

2）实现命令队列。接收方处理不过来时，可以把命令先存起来，慢慢消费。像MQ削峰其实也是这个思路，只不过更复杂。

3）远程调用场景。比如用命令对象序列化后通过网络发送，在另一端反序列化执行。Java RMI 或早期的EJB远程调用就有这种味道。

代码上关键就是四个角色：Command 接口定义 execute 和 undo，ConcreteCommand 持有 Receiver 引用并实现具体逻辑，Invoker 触发命令，Client 负责组装。

其实 Swing 的 Action 事件处理、Spring JMS Template 发送消息，底层都用了这套路。但注意别滥用，不是所有封装都要搞成命令，太简单的直接调用更清晰。

什么是合成复用原则？

合成复用原则简单说，就是优先用对象组合，而不是靠继承来复用代码。继承虽然能复用父类功能，但耦合太紧，子类依赖父类的实现细节，一旦父类变动，子类就容易出问题。

用组合的话，把功能拆成独立的组件，通过成员变量持有并委托调用，灵活性高得多。比如你有个数据库操作逻辑，别让业务类去继承一个 `BaseDao`，而是定义个 `DatabaseService`，业务类里直接引用它。

1）组合关系在运行时可以动态替换，比如换个数据源实现 2）避免继承导致的类爆炸，像 Java AWT 那种 `Button`、`TextField` 都继承自 `Component`，层次一深就搞不定 3）符合开闭原则，新增功能靠加组件，而不是改继承树

```
class OrderProcessor {
    private PaymentService payment;
    private InventoryService inventory;

    public OrderProcessor(PaymentService payment, InventoryService inventory) {
        this.payment = payment;
        this.inventory = inventory;
    }

    public void process() {
        inventory.reserve();
        payment.charge();
    }
}
```

你看这个例子，支付和库存都是组合进来的，测试时还能方便地 mock。要是全塞到继承链里，别说扩展，维护都头疼。**组合**带来的松耦合，在复杂系统里特别关键。

什么是单一职责原则？

单一职责原则讲的是，一个类或者模块应该有且只有一个引起它变化的原因。换句话说，你得让一个类只负责一块功能，别啥都往里塞。

比如你写个用户管理类，它负责处理用户信息的校验、存储，甚至还要生成报表、发邮件。这时候只要报表格式一变，或者邮件模板改了，你都得动这个类。时间一长，这个类就会变得又大又难维护，测试也搞不定。

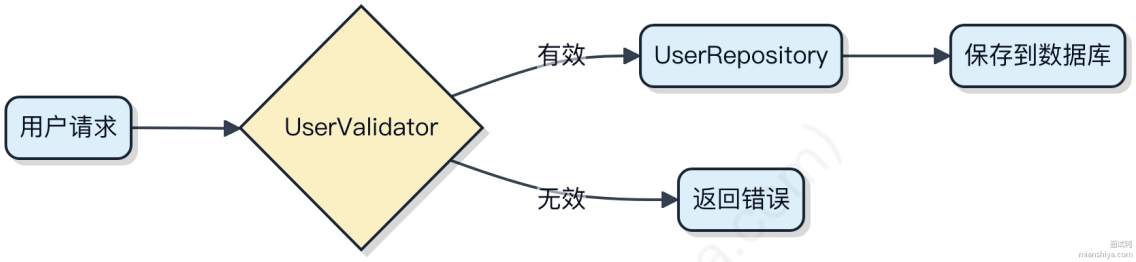
1）如果职责混杂，修改时容易牵一发而动全身
2）拆开之后，每个类专注一件事，复用性和可测试性都更强
3）团队协作也更顺畅，不同人改不同模块互不影响

举个实际例子，在 Spring 项目里，你不会把 Controller 里的业务逻辑和数据库操作全塞进一个 Service 类。通常会拆成 UserService 负责业务流程，UserRepository 负责数据存取，各自独立演化。

代码上其实很简单，就是做好边界划分：

```
class UserValidator {
    boolean isValid(User user) { ... }
}

class UserRepository {
    void save(User user) { ... }
}
```

这两个类各干各的，以后加加密逻辑也好，换数据库也好，影响范围都很清楚。很多人刚开始写代码总想“图省事”，结果后期维护成本翻倍，这就是没守住这条原则。



请用一句话概括，什么是设计模式？为什么要用？

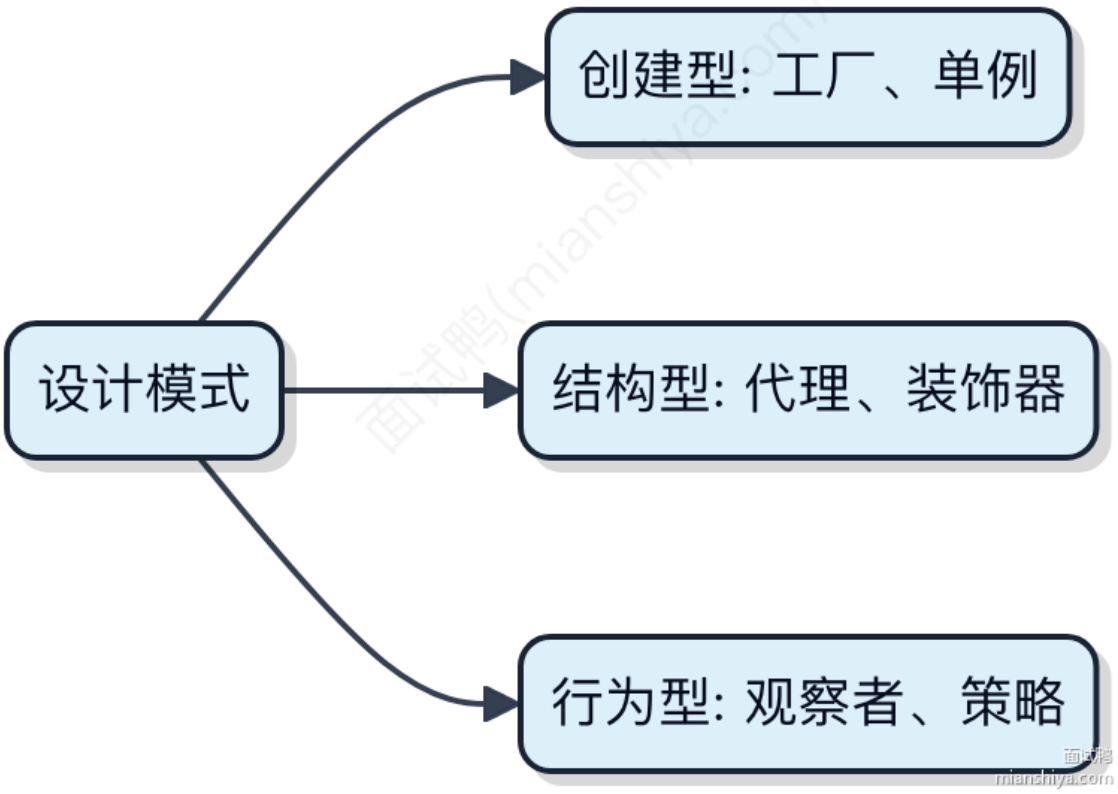
设计模式是前人总结的、针对特定软件设计问题的经典解决方案模板，它让开发者能快速复用经过验证的设计思路，避免重复造轮子。

比如你写一个数据库连接池，直接上手搞可能线程安全、资源回收都容易出问题。但如果你用了**单例模式**保证实例唯一，配合**对象池模式**管理连接生命周期，代码结构清晰，稳定性也高得多。

这些模式在 Spring、Netty 这些主流框架里到处都是。像 Spring 的 Bean 默认就是单例模式，Netty 的事件循环组也是典型的对象池实现。

用模式不是为了炫技，而是为了解决“类之间怎么协作”、“对象怎么创建”、“状态怎么管理”这类高频难题。不用的话，小项目还能靠经验顶一顶，项目一复杂就容易架构混乱，后期维护成本翻倍。

1）创建型模式解决对象怎么生出来，比如工厂模式把创建逻辑封装掉
2）结构型模式解决类和对象怎么组合，比如代理模式可以在不改原逻辑的前提下加日志、事务
3）行为型模式解决对象间怎么通信，比如观察者模式解耦事件发布与监听



什么是迪米特法则？

迪米特法则，也叫最少知道原则，说白了就是一个类应该尽量少了解其他类的内部细节，只和直接的朋友打交道。

1）每个对象都有朋友类，比如成员变量、方法参数、方法返回值里的类型，这些是允许通信的。但你不能去调用朋友的“朋友”，否则依赖就太深了。

比如你在一个服务类里拿到一个用户对象，可以调它的 getName()，但如果还要通过用户对象再去 getAccount().getBalance()，那你就依赖了 Account 类，违反了法则。正确的做法是让用户类自己提供 getBalance() 方法，把细节封装住。

2）过度遵守也可能带来问题。比如在 Spring 项目里，Service 层调用 Mapper 返回实体，再转成 DTO 返回给 Controller，这种链式调用很常见。如果硬拆，反而会让代码啰嗦。所以实际开发中要权衡，**高内聚**和**松耦合**才是目标。

3）典型应用场景是在模块划分时。像订单系统对外只暴露 OrderService，别的模块不需要知道 OrderItem、OrderLog 这些底层实现。哪怕内部结构变了，只要接口不变，外部就不受影响。

```
// 好的做法：封装内部细节
public class Order {
    private List<OrderItem> items;

    public BigDecimal getTotal() {
        return items.stream().map(Item::getPrice).sum();
    }
}

// 外部只需要调用 getTotal，不用关心 item 列表
order.getTotal();
```

别把它当成铁律，而是设计思维的引导。降低耦合度是目的，不是为了少写几个点号。

什么是装饰器模式？一般用在什么场景？

装饰器模式就是在不改变原有对象的基础上，动态地给它添加新功能。这种设计属于结构型模式，关键在于**装饰类**和被装饰的类实现同一个接口，这样外部调用者完全感知不到内部是原始对象还是被包装过的。

1）假设你有一个 `InputStream`，想让它具备缓冲能力，Java 里的 `BufferedInputStream` 就是典型例子。它接收一个 `InputStream` 实例，然后在读取时加了一层缓冲逻辑，但对外暴露的方法还是标准的 `read()`，这就是装饰器的实际应用。

2）再比如写日志或监控埋点，你可以封装一个装饰类，在每次方法调用前后打印日志，而不用去改原来的业务代码。Spring AOP 的某些实现思路也借鉴了这个模式。

```
interface Service {
    void handle();
}

class RealService implements Service {
    public void handle() { System.out.println("处理请求"); }
}
```

```java
class LogDecorator implements Service {
    private final Service target;
    public LogDecorator(Service target) { this.target = target; }
    public void handle() {
        System.out.println("开始记录日志");
        target.handle();
        System.out.println("日志记录完成");
    }
}
```

这种方式比继承更灵活。继承是在编译期静态扩展，装饰器是运行时动态叠加，想要“带缓存的功能”、“带认证的功能”，可以一层套一层地包装，拆卸也方便。

不过要注意别滥用，包装层次太多会增加调试难度，比如最终执行链路可能要穿过四五层装饰器，排查问题就比较麻烦。

什么是接口隔离原则？

接口隔离原则说白了就是让接口尽量小而专，别搞成大而全的“胖接口”。你想想，如果一个接口定义了一堆方法，但某些实现类只用其中一两个，那其他方法就成了摆设，还强迫实现类去写一堆空方法，这就很别扭。

1）接口应该只为它的使用者提供它真正需要的方法，别的都别塞进来。比如有个 `Machine` 接口，既有 `print()` 又有 `scan()` ，结果你有个类只负责打印，那它还得实现 `scan()` ，返回个不支持操作异常，这不自找麻烦嘛。

2）更合理的做法是拆开，搞个 `Printer` 接口和 `Scanner` 接口，让不同的设备去按需实现。像富士通的多功能一体机可以同时实现两个，而普通打印机就只实现 `Printer` 就行了。

3）这样拆的好处是解耦更彻底，代码维护起来也轻松。新增一个扫描功能，不影响原有的打印模块，测试范围也小。

```java
public interface Printer {
    void print();
}

public interface Scanner {
    void scan();
}

public class OfficePrinter implements Printer {
    public void print() {
        System.out.println("Printing...");
    }
}
```

客户端依赖的接口越具体，系统的灵活性和可维护性就越高。一个类不应该被强迫依赖它不用的方法，这是设计上的洁癖，也是长期演进的保障。

什么是里氏替换原则？

里氏替换原则讲的是子类应该能够替换它的父类，程序的行为不会因此出错。你用父类定义的地方，换成子类对象，系统照样能正常跑。

1）关键在于行为一致性。子类可以扩展功能，但不能改变父类原有的正确行为。比如父类的某个方法约定返回正数，子类重写后就不能返回负数，这会破坏调用方的预期。

2）常见违反场景是重写方法时抛出意外异常，或者对输入输出做不合理限制。像集合框架里 ArrayList 和 Vector 都继承自 AbstractList，它们都能替换抽象父类出现在参数、变量中，这就是遵循了该原则。

3）它和多态有关联但不是一回事。多态是语言机制，允许运行时决定调用哪个方法；而里氏替换是一种设计约束，确保这种替换是安全的。搞不定这点，继承体系就会变得脆弱，上游一改，下游全挂。

这个原则压根不关心你用了多少继承层次，只关心替换后逻辑是否成立。用的时候记住一点：别让子类“背叛”父类承诺。

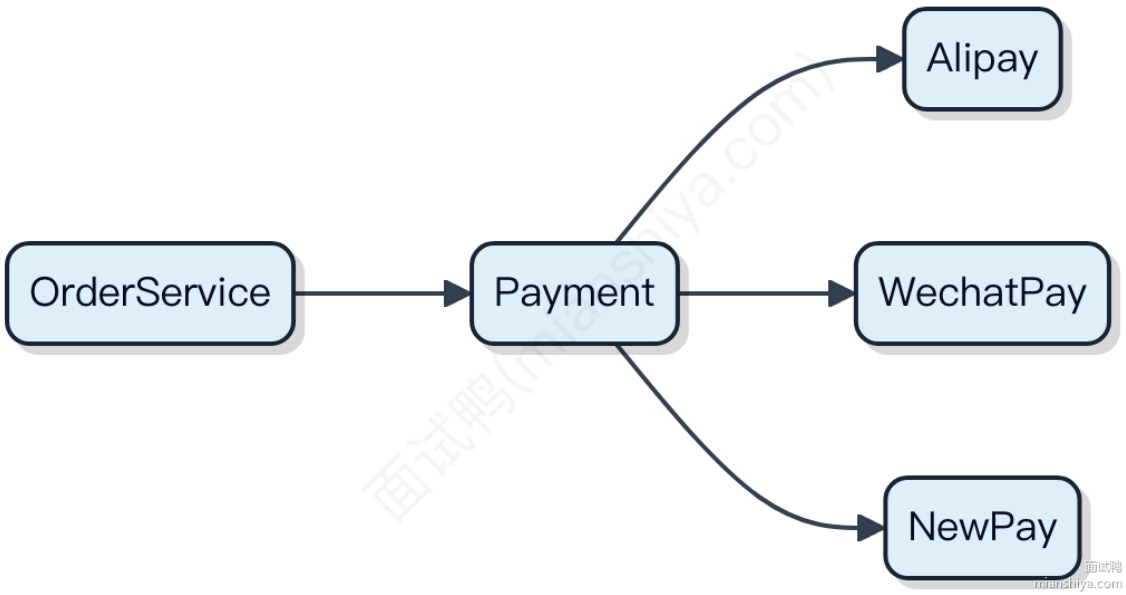
什么是开闭原则？

开闭原则讲的是软件实体应该对扩展开放，对修改关闭。意思是说，当需求变化时，你应该通过添加新代码来实现功能变更，而不是去改已经写好的、正在运行的代码。

比如你有个订单处理逻辑，现在要支持新的支付方式，理想的做法不是在原有代码里加 if-else 判断，而是通过抽象出 Payment 接口，让每种支付方式自己实现，新增时只加新类，不动旧逻辑。

这样做的好处很明显，老功能不会因为改代码而引入 bug，系统也更容易维护和测试。实际开发中，像 Spring 的拦截器链、Dubbo 的扩展点机制，都是靠这种设计支撑起来的。

1）先定义抽象，比如接口或抽象类
2）具体行为由子类或实现类完成
3）新增功能时只增加新实现，不碰原有分支



这个模式用得好的话，核心流程稳定，新功能靠插件式接入。其实很多设计模式，像策略、装饰器、模板方法，本质上都在帮我们满足这条原则。

什么是备忘录模式，一般用在什么场景？

备忘录模式解决的是对象状态的保存和恢复问题，不破坏原封装性。它通过一个中间角色把对象的内部状态抽出来存到外部，需要时再塞回去。

1）核心是三个角色：原发器（Originator）是那个有状态要保存的对象，比如一个文本编辑器。负责人（Caretaker）负责管理备忘录，但不能看内容。备忘录（Memento）就是个数据容器，存原发器的状态快照。

2）典型场景是撤销操作。比如你用记事本写东西，Ctrl+Z 撤销上一步，背后其实就是把之前存的备忘录拿回来覆盖当前状态。游戏存档也是类似逻辑，打不过关卡就读档重来。

3）另一个常见用途是事务回滚。像 Spring 的 `@Transactional` 注解，万一事务失败，可以通过保存的旧状态来回滚。虽然 Spring 不一定直接用备忘录模式实现，但思想是一致的。

代码上很简单，原发器提供 `save()` 和 `restore(Memento m)` 方法，备忘录类通常设计成原发器的内部类，这样能访问私有成员。

```
class Editor {
    private String content;

    public Memento save() {
        return new Memento(content);
    }

    public void restore(Memento m) {
        content = m.getContent();
    }

    public static class Memento {
        private final String content;
        private Memento(String content) { this.content = content; }
        private String getContent() { return content; }
    }
}
```

这个模式的好处是隔离了状态存储逻辑，原发器不用关心怎么存，负责人也不用知道存了啥。但频繁创建备忘录会吃内存，尤其是状态大又频繁操作的场景，得考虑做快照间隔或者序列化压缩。

什么是访问者模式？一般用在什么场景？

访问者模式解决的是数据结构和操作逻辑解耦的问题。当一个数据结构包含多种类型对象，而你需要对这些对象执行不同的操作，并且这些操作可能会频繁增减时，直接在类里加方法会让代码越来越臃肿。

它把操作“外移”，让外部逻辑通过 **访问者** 的方式来处理各个元素。元素负责接受访问者，调用其对应的方法，反过来触发具体行为。

1） 元素接口定义 `accept(Visitor)` 方法
2） 每个具体元素实现 `accept` ，传入自身引用调用访问者的 `visit(this)`
3） 访问者接口声明多个 `visit` 重载方法，对应不同元素类型
4） 具体访问者实现这些方法，封装实际逻辑

比如编译器 AST（抽象语法树）遍历，不同节点（变量、循环、函数）需要做类型检查、代码生成、优化等操作。如果每个节点都内置这些方法，维护成本极高。用访问者模式，每种功能写一个访问者类，新增优化策略也不用动节点代码。

另一个典型场景是报表生成，比如 Spring Data JPA 中的查询树处理，或者 Jackson 反序列化时的节点遍历逻辑。

这种模式适合稳定的数据结构 + 多变的操作。但一旦元素类型变了，所有访问者都要改，反而更麻烦。所以前提是结构相对固定。

什么是责任链模式？一般用在什么场景？

责任链模式就是把多个处理器串成一条链，请求沿着链往下传，直到某个处理器处理它为止。每个节点只关心自己能不能处理，能就办了，不能就丢给下一个，调用者不用管背后是谁干的。

这种模式的关键在于解耦。你往链上加新处理器，完全不用动原来的代码，符合开闭原则。比如在 Web 开发里，过滤器（Filter）就是典型的责任链，像 Spring Security 的安全过滤器链，一堆 Filter 依次执行，有的做认证，有的做授权，漏掉一个都不行。

再比如电商场景下的审批流，报销申请先由直属领导审，然后财务审，金额超过 1 万还得总监过一遍。每层只判断自己该不该拦，不用知道前后是谁。

写法上一般让处理器实现同一个接口，里面有个 next 引用指向下一个处理器，处理方法里先判断自己是否处理，否则调用 next.handle()。

```
public abstract class Handler {
    protected Handler next;
    public void setNext(Handler next) { this.next = next; }
    public abstract void handle(Request request);
}

public class Level1Handler extends Handler {
    public void handle(Request request) {
        if (request.getType() == "type1") {
            System.out.println("Level1 处理");
        } else if (next != null) {
            next.handle(request);
        }
    }
}
```

注意别搞成死循环，还有要考虑是否要支持多路径或并行处理，原生责任链是线性的。有些框架像 Netty 的 ChannelPipeline，也是责任链，但更复杂，支持入站出站双向传播。

什么是策略模式？一般用在什么场景？

策略模式解决的是“一堆 if-else 或 switch 分支里塞不同算法”的问题。你想啊，比如做支付功能，微信支付、支付宝、银联，每种逻辑完全不同，要是全写在一个方法里，代码立马爆炸，维护起来根本搞不定。

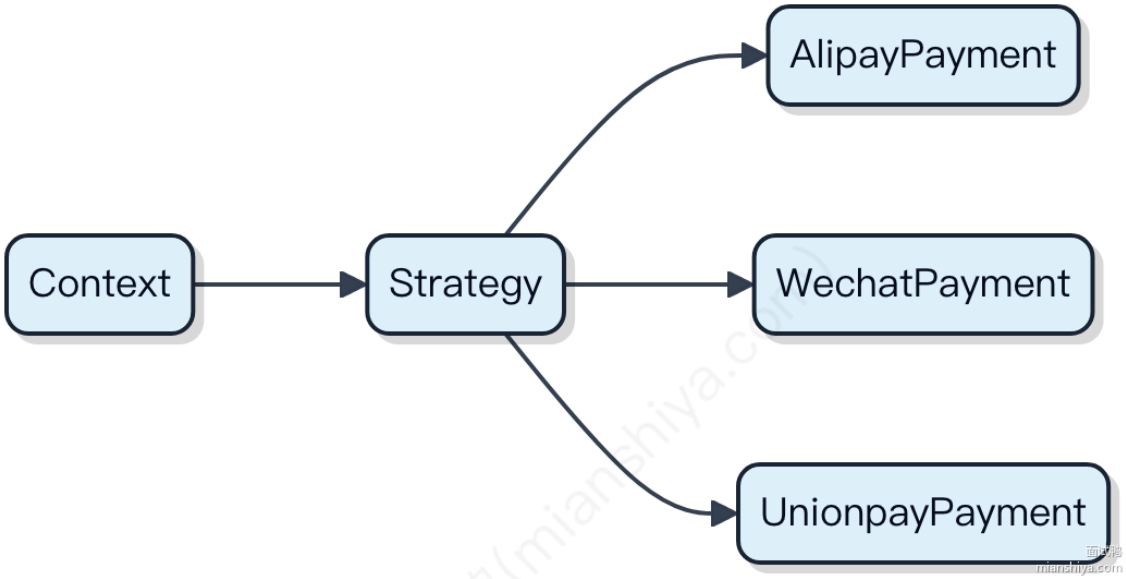
它的核心是把每个算法封装成独立的类，都实现同一个接口。运行时根据条件选一个具体实现来用。这样新增支付方式不用改老代码，符合开闭原则。

一般在这些场景特别好使：1）有明显的行为分支，比如计算运费，按城市、重量、会员等级有不同算价规则 2）需要动态切换算法，像导出报表支持 CSV、Excel、PDF 三种格式 3）避免重复的条件判断，像促销活动满减、打折、返券

举个例子，Spring 的 `Resource` 接口就是典型，`ClassPathResource`、`FileSystemResource`、`UrlResource` 各自处理不同路径类型的资源加载，调用方压根不经过具体实现。

```java
public interface Strategy {
    void execute();
}

public class Context {
    private Strategy strategy;

    public void setStrategy(Strategy strategy) {
        this.strategy = strategy;
    }

    public void run() {
        strategy.execute();
    }
}
```

实际项目里，配合 Spring 的 IoC，直接把所有策略实现注入进来，用 map 存好，key 是策略名，取的时候 `strategies.get("alipay").execute()` 就行，干净利落。



什么是状态模式？一般用在什么场景？

状态模式本质上是把对象的行为包装在不同状态类里，让对象在运行时通过切换状态类来改变行为。你不用写一堆 if-else 判断当前状态再执行对应逻辑，而是让“状态”自己决定该干什么。

1）每个状态都实现统一接口，比如 `handle()` 方法，具体行为由子类实现
2）上下文（Context）持有一个状态实例，所有请求都委托给当前状态处理
3）状态之间可以互相转换，比如订单从“待支付”变成“已发货”

这种模式特别适合状态多、状态转换复杂、每个状态行为差异大的场景。举个例子，电商系统的订单状态机——待支付、已支付、已发货、已完成、已取消，如果全用 if-else 堆在一起，改一个状态跳转就得动整个方法，后期根本搞不定。用状态模式，每个状态独立封装，新增状态或修改流转只影响对应类。

游戏开发里也常见，比如角色有“ idle ”、“running”、“attacking”状态，不同状态下按键响应完全不同，用状态模式就很清晰。

```
interface OrderState {
    void pay(OrderContext ctx);
    void ship(OrderContext ctx);
}

class PendingPayment implements OrderState {
    public void pay(OrderContext ctx) {
        ctx.setState(new Shipped());
    }
    public void ship(OrderContext ctx) { /* 不允许操作 */ }
}
```

状态模式真正发力的点在于它让状态变更和行为扩展变得可控。你不再是在一个大类里修修补补，而是用多态代替条件判断，代码结构更稳定。

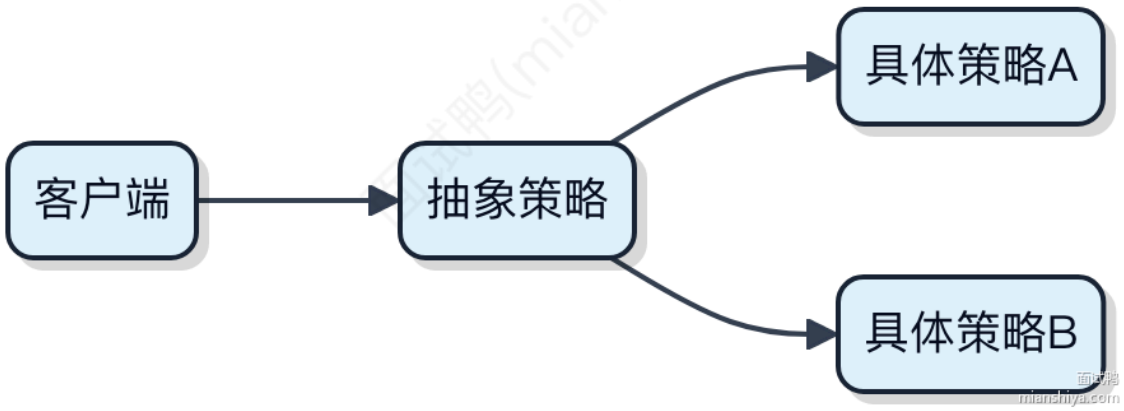
什么是设计模式？请简述其作用。

设计模式不是语法也不是框架，它是前人总结出的、针对特定问题的**可复用解决方案模板**。本质上是代码结构和组织方式的经验结晶。

1）解决重复出现的设计问题
比如对象创建太散乱，就用工厂模式统一出口；系统耦合太紧，就引入观察者模式解耦。Spring 里的 BeanFactory 就是简单工厂的体现，MyBatis 的插件机制用了责任链。

2）提升代码可维护性和扩展性
以策略模式为例，不同算法独立封装，新增逻辑不用改原有 if-else。电商项目里计算运费、优惠规则基本都这么干。

3）提供通用语言
团队沟通时说“这里用模板方法”，比解释“父类定流程子类重步骤”更高效。就像说“MVC”大家立刻明白分层结构。



注意别过度设计。小项目硬套 23 种模式反而搞复杂了。GOF 提出的分类（创建型、结构型、行为型）可以作为学习路线图，但实战中按需选用才是正道。

什么是模板方法模式？一般用在什么场景？

模板方法模式说白了就是把一个算法的流程固定下来，把不变的部分在抽象类里写死，变的部分交给子类去实现。父类控制整体步骤，子类只负责具体细节。

1） 定义了一个算法骨架，比如处理请求的流程是：校验 -> 处理 -> 记录日志。这个流程不能改。

2） 把校验、处理这些步骤声明成抽象方法，子类根据业务实现自己的逻辑。比如支付流程里，微信支付和支付宝支付的处理方式不同，但整体流程一致。

3） 钩子方法可以用来做条件分支，父类通过 `isDebug()` 这样的默认返回值控制是否打印日志，子类可选择性覆盖。

这种模式在框架设计里特别常见。Spring 的 `JdbcTemplate` 就是个典型例子，你写 SQL，它帮你搞定连接管理、异常转换、资源释放这些脏活累活。还有像 `AbstractList` 里的 `addAll` 方法，底层调用 `add` ，而 `add` 是抽象的，留给具体集合实现。

代码长这样：

```
abstract class ProcessTemplate {
    public final void execute() {
        validate();
        process();
        log();
    }

    abstract void validate();
    abstract void process();

    void log() { System.out.println("default log"); }
}

class MyProcess extends ProcessTemplate {
    void validate() { /* 自定义校验 */ }
    void process() { /* 业务处理 */ }
}
```

适合用在多个子类有相同工作流程，只是细节不同的场景。要是每个子类流程都不一样，硬套模板反而更麻烦。

什么是观察者模式？一般用在什么场景？

观察者模式解决的是一对多依赖关系，当一个对象状态改变，所有依赖它的对象都能自动收到通知。这种模式在事件驱动系统里特别常见。

1）被观察的对象叫主题（Subject），它维护一个观察者列表。状态变更时，遍历列表调用每个观察者的更新方法。
2）观察者实现统一接口，只需定义如何响应变化，不需要主动去拉数据。

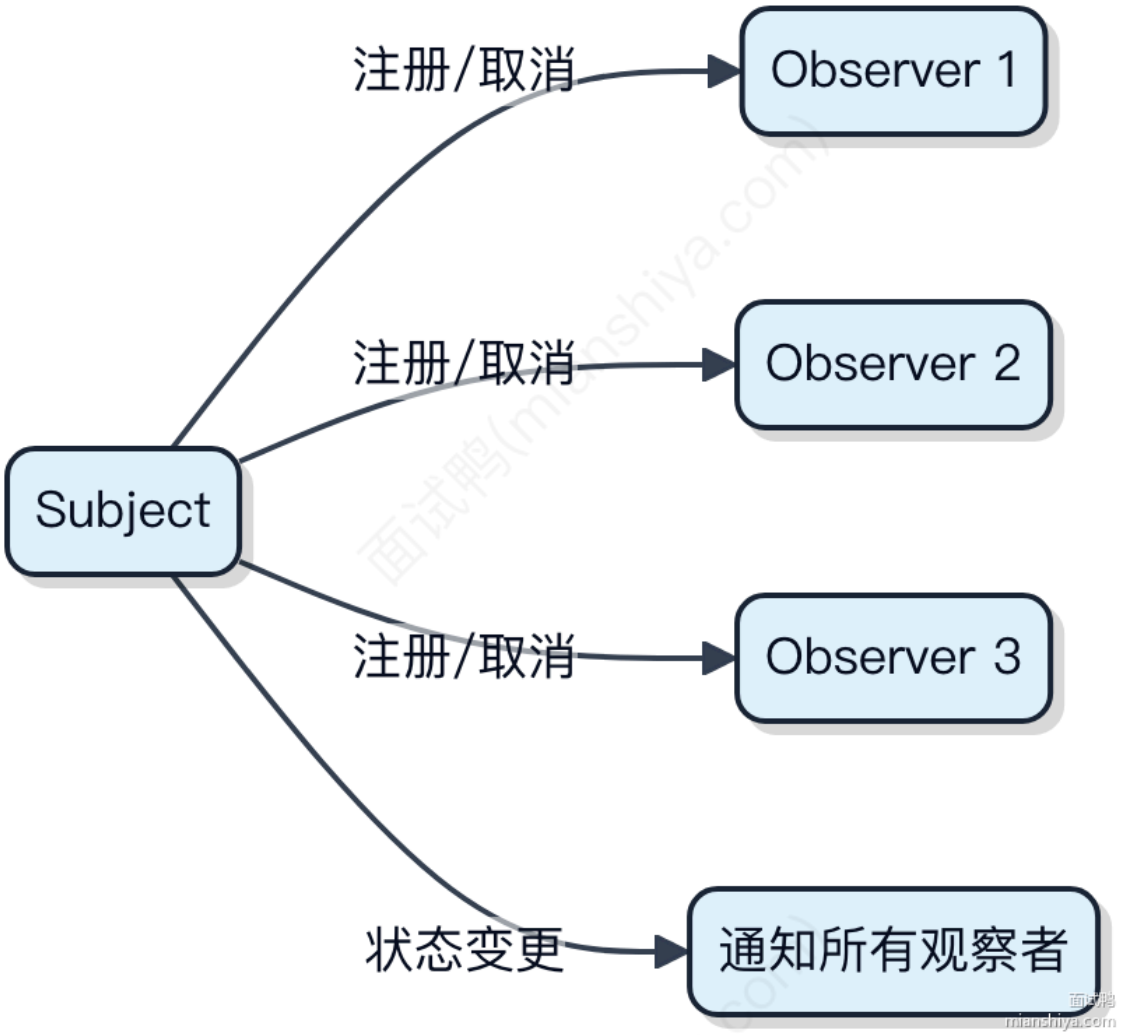
这种设计把通知逻辑收拢到主题内部，避免散落在各处。像 Spring 的 `ApplicationEvent` 机制就是典型应用，发布一个事件，多个监听器自动触发。Swing 的按钮点击、前端框架的事件绑定也都是这个思路。

代码上其实很简单：

```java
interface Observer {
    void update(String message);
}

class Subject {
    private List<Observer> observers = new ArrayList<>();
    private String state;

    public void setState(String state) {
        this.state = state;
        notifyObservers();
    }

    private void notifyObservers() {
        observers.forEach(o -> o.update(state));
    }
}
```

适合用在业务解耦的场景，比如订单创建后要发短信、更新库存、记录日志。把这些操作做成观察者，主流程只管发事件，不用硬编码后续动作。但要注意别滥用，观察者太多会搞不清执行链路，调试起来头疼。

面试鸭(mianshiya.com)



[请解释什么是单例模式，并给出一个使用场景](#)

单例模式保证一个类在整个应用中只有一个实例，并提供一个全局访问点。这种模式属于创建型设计模式，核心是控制实例的创建时机和数量。

实现上通常把构造函数私有化，通过静态方法获取唯一实例。重点在于线程安全和防止反射破坏，比如饿汉式在类加载时就创建，简单但可能浪费资源；懒汉式延迟加载，但得加锁。

1）饿汉式写法最稳妥，JVM 保证线程安全

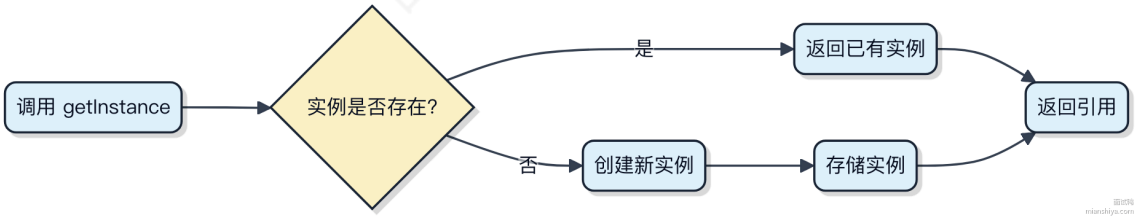
```
public class Singleton {
    private static final Singleton INSTANCE = new Singleton();
    private Singleton() {}
    public static Singleton getInstance() {
        return INSTANCE;
    }
}
```

2）懒加载推荐使用静态内部类，既延迟又安全

```
public class Singleton {
    private Singleton() {}
    private static class Holder {
        static final Singleton INSTANCE = new Singleton();
    }
    public static Singleton getInstance() {
        return Holder.INSTANCE;
    }
}
```

常见场景是配置管理器或线程池。像 `java.lang.Runtime` 就是个典型例子，每个 JVM 进程只需要一个运行时环境，通过 `getRuntime()` 获取唯一实例来执行系统级操作。

注意序列化和反射可能破坏单例，如果类实现了 `Serializable` ，要重写 `readResolve()` 方法返回同一实例。



请描述简单工厂模式的工作原理。

简单工厂模式的核心是把对象的创建过程集中到一个“工厂”类里，客户端不用关心具体怎么创建实例，只要告诉工厂想要什么类型就行。

1） 工厂类内部通常有一个静态方法，接收一个参数比如类型标识，通过 if-else 或 switch 判断该返回哪个具体实现。比如传入 "mysql" 就返回 `MySqlDriver` ，传入 "redis" 就返回 `RedisClient` 。

2） 客户端代码只需要依赖抽象接口或父类，不直接调用 new。这样新增产品时，理论上只需修改工厂逻辑，而不必改动所有使用方。

3） 这种模式其实违背了开闭原则，每加一种新产品就得改工厂方法。所以适合产品种类固定、变化少的场景，比如配置解析器，支持 JSON 和 XML 两种格式就够了。

代码示例如下：

```
public class ParserFactory {
    public static Parser create(String type) {
        if ("json".equals(type)) return new JsonParser();
        if ("xml".equals(type)) return new XmlParser();
        throw new IllegalArgumentException("Unknown type");
    }
}
```

说白了，它就是个封装了 new 的工具类，让调用方更干净，但扩展性有限。真要搞灵活，还是得上 **工厂方法模式** 或 **抽象工厂**。

什么是建造者模式？一般用在什么场景？

建造者模式解决的是复杂对象构建过程的封装问题，尤其当构造函数参数太多、对象创建需要多个步骤时特别有用。你肯定遇到过那种一串 new 的时候传七八个参数，还分不清哪个是哪个，这时候就该上建造者了。

1）它把对象的构建逻辑从构造中剥离出来，通过链式调用一步步设置属性，最后 build 一下生成最终对象。代码可读性直接提升一个档次。

2）典型的使用场景就是像 HTTP 请求构建、数据库连接配置、DTO 转换这类需要灵活组合字段的地方。比如 OkHttp 的 Request 就用了建造者，你可以先 setUrl，再 addHeader，最后 build 出请求对象，每步都清晰明了。

3）和工厂模式不同，建造者关注的是**同一个产品族内部的复杂构建流程**，而不是创建哪种类型的产品。工厂决定“生产什么”，建造者决定“怎么组装”。

代码长这样：

```
User user = User.builder()
    .name("zhangsan")
    .age(25)
    .email("zhangsan@example.com")
    .build();
```

你看这比 new User(null, "zhangsan", 25, null, "zhangsan@example.com", ...) 强太多了，参数多了也不会搞不定。而且可以设置默认值，某些必填字段还能在 build 阶段做校验，不满足直接抛异常。

Spring Boot 的 RestTemplate、Jackson 的 ObjectMapper 配置也都用了类似思路。这种模式在构建不可变对象（Immutable）时尤其有价值，因为所有字段都是 final 的，只能靠 builder 一次性赋值。

什么是原型模式？一般用在什么场景？

原型模式就是通过拷贝一个已有对象来创建新对象，而不是走构造函数那套流程。关键在于实现 `Cloneable` 接口并重写 `clone()` 方法，让对象能自己复制自己。

1）当你发现创建对象的过程特别重，比如要查数据库、读文件、网络请求，这时候用原型模式缓存一个“样板”对象，后续直接克隆，性能提升非常明显。像 Spring 框架里的 prototype 作用域，每次 `getBean()` 都返回一个新实例，底层就是靠克隆机制实现的。

2）在某些配置中心场景里，比如你有一个默认配置对象，每次新增一个服务实例时只需要微调几个字段，其他都一样，这时候基于默认配置 clone 一份再改，比重新 new 一套要安全又高效。

3）注意浅拷贝和深拷贝的问题。`Object.clone()` 默认是浅拷贝，如果对象里有引用类型，克隆后的对象还是指向同一个引用，改一个另一个也跟着变。真要隔离就得自己实现深拷贝，比如通过序列化反序列化，或者递归拷贝成员对象。

```
public class Config implements Cloneable {
    private String name;
    private Map<String, String> settings;

    @Override
    public Config clone() {
        try {
            Config cloned = (Config) super.clone();
```

```
            // 手动深拷贝引用类型
            cloned.settings = new HashMap<>(this.settings);
            return cloned;
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException(e);
        }
    }
}
```

这种模式适合那种“创建成本高、结构复杂但差异小”的对象创建场景，属于典型的以空间换时间。

什么是适配器模式？一般用在什么场景？

适配器模式解决的是接口不兼容问题，你手里的类和系统期望的接口对不上，又不能改源码，这时候就需要一个中间层来做转换。

比如你有一个旧的支付服务 `LegacyPayment` ，它提供 `makePayment` 方法，但新订单系统要求实现 `ModernPayment` 接口的 `pay` 方法。直接用肯定不行，重构成本又高。适配器就在这儿起作用，它实现新接口，内部包装旧服务，把 `pay` 的调用转成对 `makePayment` 的调用。

1）写一个适配器类实现目标接口
2）构造时传入被适配的对象
3）在方法里做逻辑映射

```
class PaymentAdapter implements ModernPayment {
    private LegacyPayment legacy;

    public PaymentAdapter(LegacyPayment legacy) {
        this.legacy = legacy;
    }

    @Override
    public void pay(double amount) {
        legacy.makePayment((int) amount); // 类型转换、参数调整都在这
    }
}
```

常见场景有：集成第三方库版本冲突，像把 Redis 2.x 客户端适配到统一的 3.x 接口；还有 Spring MVC 里的 HandlerAdapter，不同类型的控制器方法能共存，靠的就是适配器统一调用方式。

这种模式的核心是**不改变原有代码的前提下完成接口匹配**，属于结构型设计模式。注意别滥用，如果系统全是适配器，说明架构本身可能就有问题。

什么是桥接模式？一般用在什么场景？

桥接模式的关键是把**抽象**和**实现**解耦，让它们能独立变化。比如你有个图形类，既要支持不同形状（圆形、方形），又要支持不同渲染方式（Windows、Linux）。如果用继承硬写，组合爆炸，8个类都搞不定。

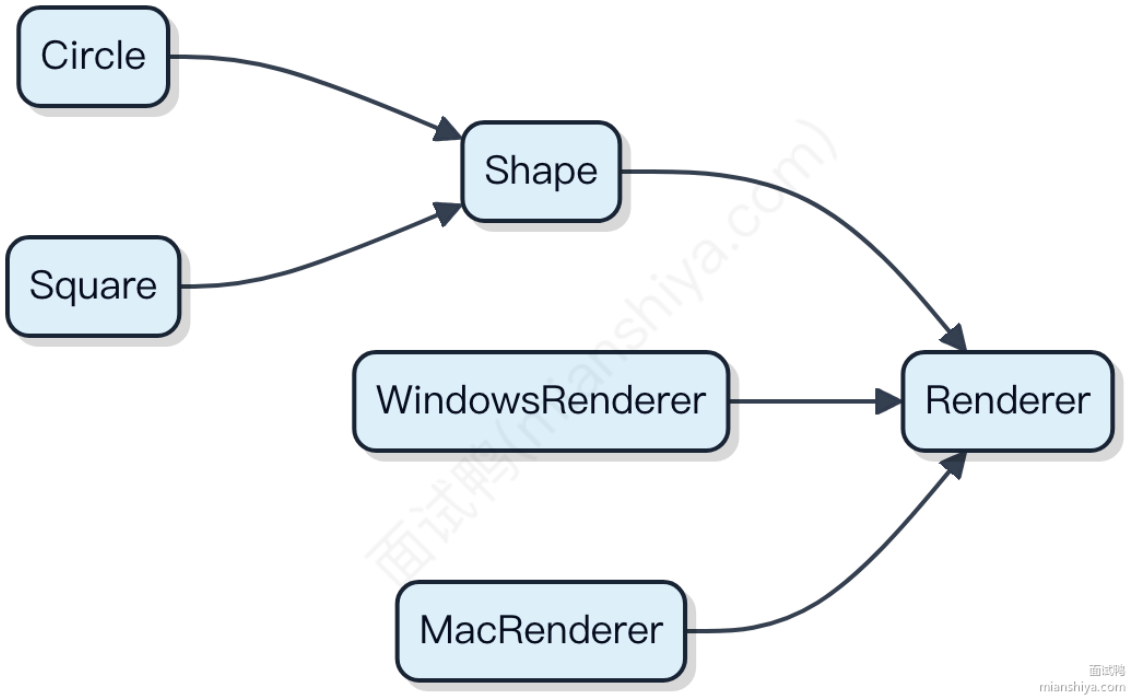
1） 把变化维度拆开。形状作为抽象层，渲染作为实现层，两者通过组合连接。 2） 抽象层持有一个实现层的接口引用，运行时动态绑定具体实现。 3） 新增形状或新渲染器，都不用动对方代码，符合开闭原则。

典型场景是跨平台 GUI 库。Swing 里的 `Component` 和 `LookAndFeel` 就是桥接——同一个按钮，在 Windows 和 macOS 上外观不同，但行为一致。还有 JDBC 驱动，`Connection` 接口是抽象，MySQL、Oracle 驱动是具体实现，切换数据库不改业务代码。

```
interface Renderer {
    void render(String shape);
}

class WindowsRenderer implements Renderer {
    public void render(String shape) {
        System.out.println("Draw " + shape + " on Windows");
    }
}

abstract class Shape {
    protected Renderer renderer;
    public Shape(Renderer renderer) {
        this.renderer = renderer;
    }
    abstract void draw();
}

class Circle extends Shape {
    public Circle(Renderer renderer) {
        super(renderer);
    }
    void draw() {
        renderer.render("Circle");
    }
}
```

实际调用时，new Circle(new WindowsRenderer())，就能画出一个 Windows 风格的圆。换 Mac 渲染器，只要换参数，形状类完全不动。

面试鸭(mianshiya.com)



什么是组合模式？一般用在什么场景？

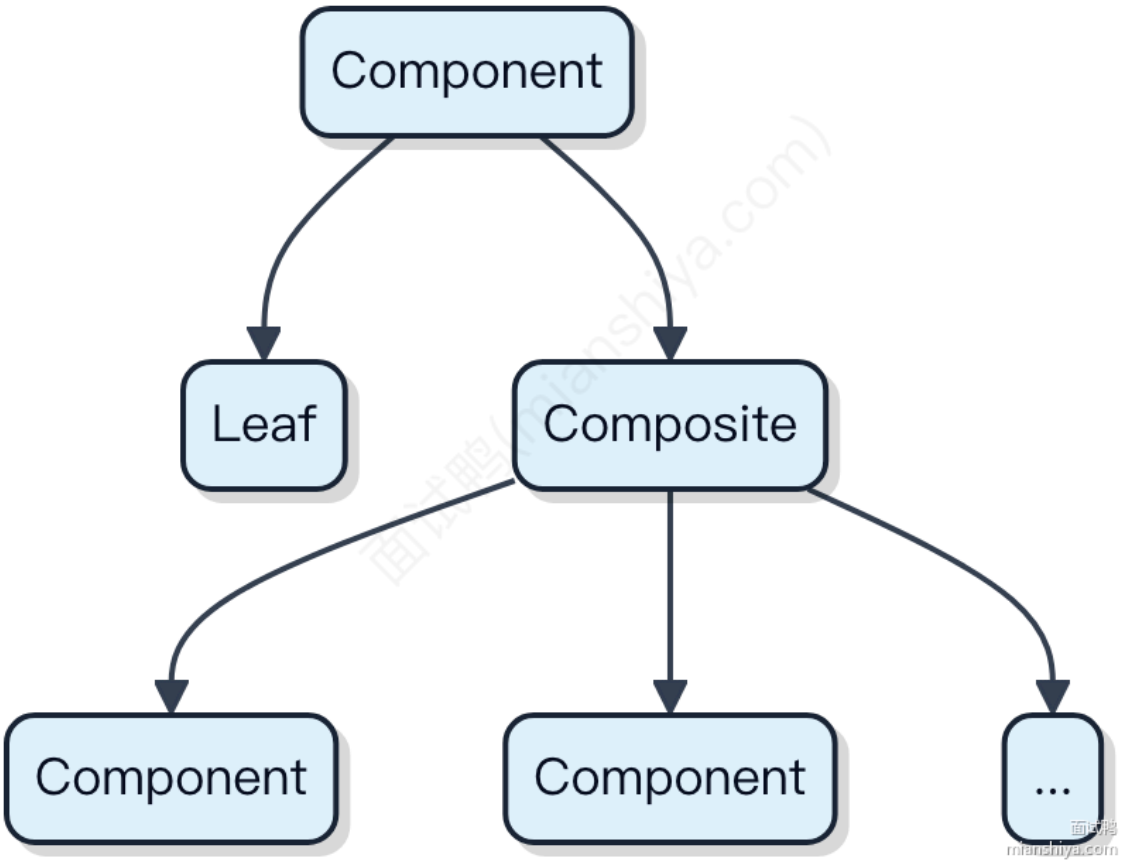
组合模式解决的是树形结构的统一处理问题。你有一堆对象，它们之间是父子关系，形成一个树，你想让客户端不管是操作根节点、分支还是叶子，都用同一套接口。

1）它把对象组织成树形结构，每个节点要么是容器（组合节点），要么是叶子节点。关键是，容器和叶子都实现同一个抽象接口，这样上层调用者就不需要关心当前操作的是不是还有子节点。

2）典型场景比如文件系统，目录可以包含文件和其他目录，但你可以对它们统一调用 `getSize()` 或 `delete()` 。再比如 UI 组件，窗口包含面板，面板包含按钮，但所有组件都能响应 `draw()` 调用。

3）另一个常见例子是菜单系统。像 Swing 或 JavaFX 中的菜单项和子菜单，都可以被点击或渲染，组合模式让遍历和操作变得简单。

代码核心就是一个 `Component` 接口，定义通用行为，然后 `Composite` 里维护一个 `List<Component>` 子节点列表，调用时递归下去。



这种模式的好处是扩展性强，加新类型的节点不影响原有逻辑。但缺点也很明显，如果节点类型差异大，强行统一接口会让很多方法在某些子类里空实现，有点别扭。用的时候得看场景是否真的具备树形 + 统一操作的特征。

什么是外观模式？一般用在什么场景？

外观模式本质是给复杂子系统加一层统一的门面，让外部调用更简单。你不用关心背后一堆模块怎么协作，只通过一个外观类就能搞定。

1）假设你的系统里下单要走订单、库存、支付、通知四个服务，每个都有自己的 API 和调用顺序。如果每个业务都自己拼这些逻辑，代码会散落各处，改一个流程就得动好几处。

2）这时候搞个 `OrderFacade` ，它内部封装了完整的下单流程，暴露一个 `placeOrder()` 方法。上层应用只需要调这个方法，压根不经过底层那堆服务。

3）这种模式特别适合中台服务或者通用能力平台。比如你在公司做了一个统一的消息网关，对接了短信、邮件、钉钉、企业微信多个渠道，业务方不需要知道每个渠道的细节，调一个 `sendMessage()` 就行，背后由外观类去路由和处理。

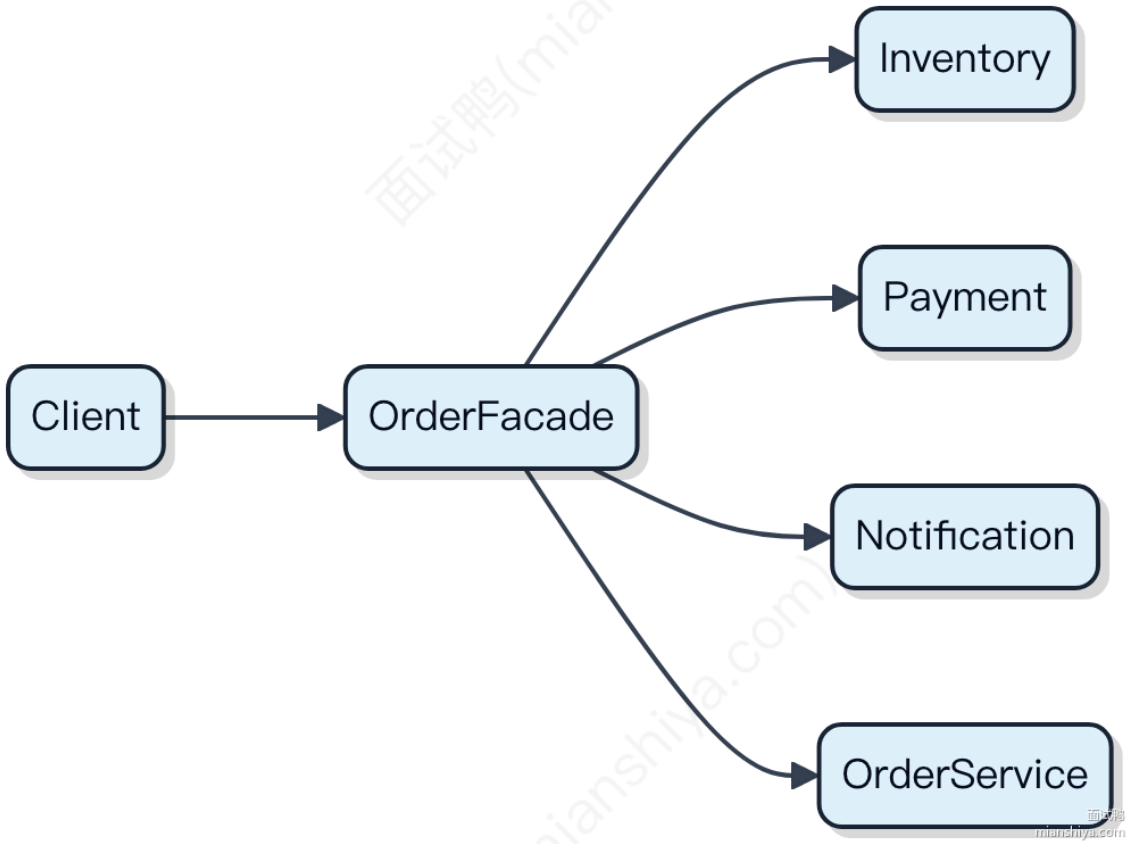
代码上看很简单：

```
class OrderFacade {
    public void placeOrder(Order order) {
        inventoryService.reduce(order);
        paymentService.pay(order);
```

```
        notificationService.send(order);
        orderService.save(order);
    }
}
```

常见于重构老旧系统时引入，把混乱的调用关系收拢。也常用于 API 层对内隐藏微服务细节，比如用 Spring Cloud Gateway 或者自定义 Facade 服务对外暴露简洁接口。

注意它不是代理，不控制访问过程，也不做增强，就是单纯地简化调用。要是你发现某个功能需要组合多个服务才能完成，而且这个组合逻辑重复出现，那就是外观模式的典型场景。



什么是迭代器模式？一般用在什么场景？

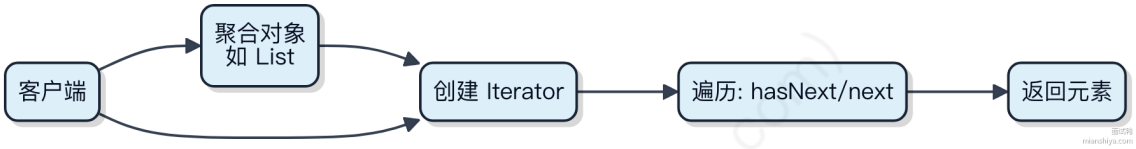
迭代器模式说白了就是把集合内部的遍历逻辑封装起来，让调用者不用关心底层是数组、链表还是树，统一用 `next()` 、 `hasNext()` 这一套方式去访问元素。

1）它最核心的价值在于**解耦**。比如你写了个 ArrayList，别人用的时候如果直接操作内部数组，那以后你换成链表结构，他的代码就全得改。但用了迭代器，接口不变，底层爱怎么变怎么变。

2）Java 里的 `Collection` 框架全靠它撑着。List、Set、Map 的遍历，像 `for (String s : list)` 这种增强 for 循环，背后都是 `Iterator` 在干活。你用 LinkedList 或 HashSet，表面写法完全一样。

3）适用场景很明确：只要你在设计一个容器类，或者需要对外提供遍历功能，就应该考虑返回一个迭代器。典型例子像 MyBatis 查询结果的 `ResultSet` 遍历，或者 Kafka Consumer 拉取的消息集合，都用了类似思想。

不适用的情况也有，比如数据量特别大且需要随机访问的场景，用下标可能更高效，这时候硬套迭代器反而拖慢性能。



别小看这个模式，它把遍历的“脏活累活”藏起来了，别人用起来只觉得顺滑，这其实就是好 API 的设计直觉。

什么是中介者模式？一般用在什么场景？

中介者模式解决的是多个对象之间**交互复杂**的问题。当一堆组件直接互相调用，改一个可能牵出一堆问题，维护起来特别费劲。

它的做法是把原本分散的交互逻辑集中到一个“中间人”身上。所有组件不再彼此通信，而是只和这个中介者打交道。比如聊天室场景，用户不直接发消息给对方，而是发给房间，由房间负责转发。

1）组件之间解耦，各自只依赖中介者，修改影响范围小
2）交互逻辑集中在一处，方便控制和扩展
3）中介者本身可能变复杂，但换来的是整体结构清晰

典型应用像 GUI 界面里多个控件联动。比如下单页面，选城市影响运费，选优惠券影响总价，库存不足又要禁用按钮。要是每个控件都监听其他控件，代码很快乱成一锅粥。用中介者统一处理这些规则，逻辑就清爽多了。

```
public class OrderMediator {
    private CitySelector city;
    private CouponBox coupon;
    private PriceDisplay price;
    private SubmitButton submit;

    public void onCityChanged() {
        price.update(calculate(city.get(), coupon.get()));
    }

    public void onCouponChanged() {
        price.update(calculate(city.get(), coupon.get()));
        submit.setEnabled(enoughStock(coupon.get()));
    }
}
```

这种模式在游戏开发、工作流引擎里也常见。比如 Unity 的事件系统，或者 Spring 中的 ApplicationEventPublisher，其实都有中介者的影子。

装饰器、适配器、代理、桥接这四种设计模式有什么区别？

这四个模式确实容易混，关键得看它们解决的问题和结构差异。

装饰器是给对象动态加功能，不改原类，也不影响其他实例。比如 Java 的 `BufferedInputStream(InputStream)`，就是套一层增加缓冲能力，调用者无感知。核心是“透明增强”，所有

装饰器和被装饰的类实现同一接口，层层包裹。

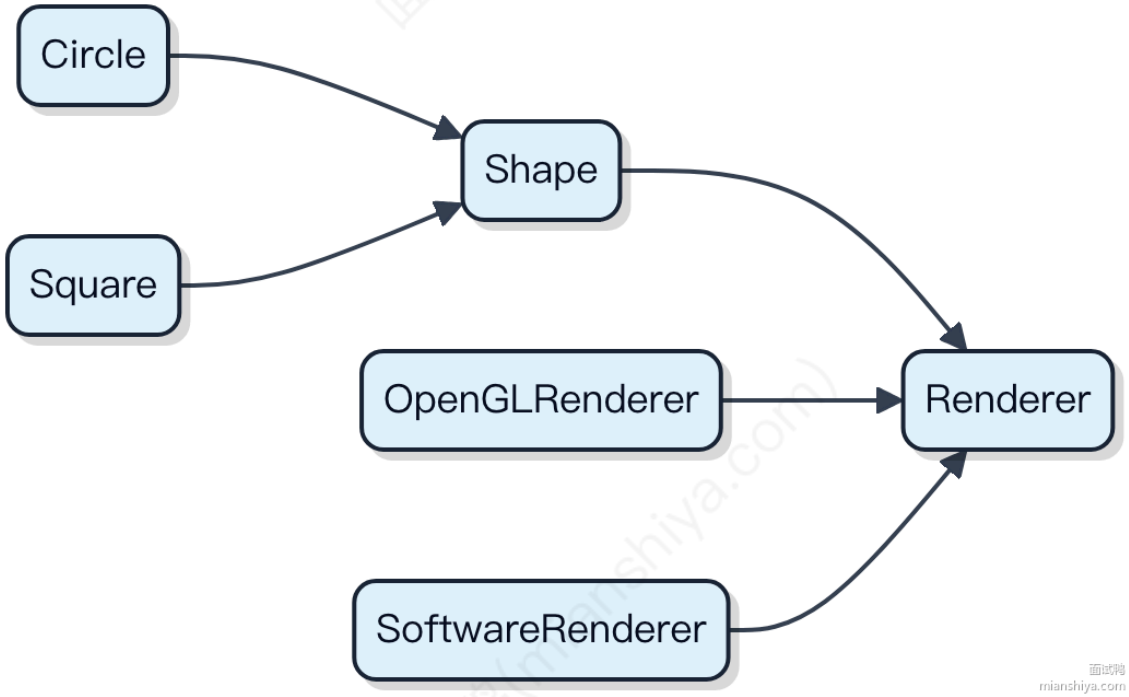
```
new BufferedInputStream(new FileInputStream("a.txt"));
```

适配器是让接口不兼容的类能一起工作。比如老系统返回 List，新服务要 Stream，写个适配器转一下。它不改原有逻辑，只是做接口转换，像电源插头转接器一样。

代理控制对目标对象的访问，本身和目标实现同一接口。比如 RPC 调用里，客户端拿到的是个本地 **代理** 对象，实际请求被拦截后发到远程。Spring AOP 的 JDK 动态代理也是这个路子，代理干的是访问控制、日志这些脏活累活。

1）装饰器关注功能叠加
2）适配器关注接口兼容
3）代理关注访问控制

桥接是把抽象和实现解耦，让两者能独立变化。比如不同形状（圆、方）可以搭配不同渲染方式（抗锯齿/非抗锯齿），用继承会爆炸，桥接通过组合搞定。JDBC 驱动就是典型，应用代码不依赖具体数据库实现。



桥接在框架底层用得多，业务代码少见。前三种在日常开发里天天见。

什么是享元模式？一般用在什么场景？

享元模式本质是对象的缓存复用，把可共享的状态抽出来，避免重复创建。它适合大量相似对象的场景，通过共享减少内存占用。

比如字符串常量池、Integer 的 -128 到 127 缓存，都是典型的享元应用。你 new Integer(100)，JVM 不会每次都新建，而是从缓存数组里拿已有的实例。

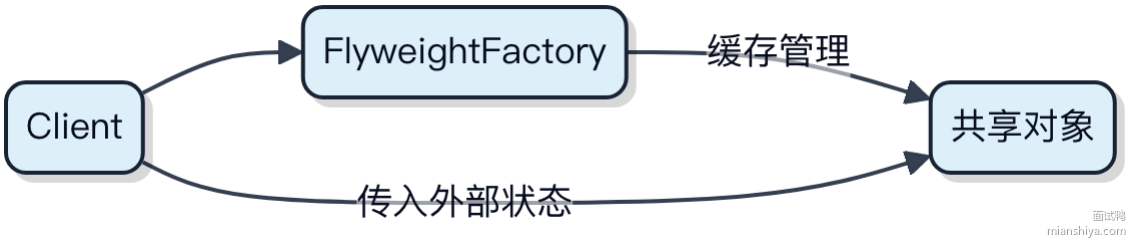
这种模式关键在于区分内部状态和外部状态。内部状态是能共享的，比如字符内容；外部状态是变化的，得由客户端传入，不能放在享元对象里。

1）内部状态不可变，通常在构造时确定
2）外部状态每次调用时传入，不作为对象属性存储
3）需要一个工厂管理享元对象的创建和缓存

```
public class CharacterFlyweight {
    private final char symbol; // 内部状态

    public void display(int fontSize, String fontColor) { // 外部状态作为参数
        System.out.println("显示 " + symbol + ", 大小: " + fontSize + ", 颜色: " +
fontColor);
    }
}
```

典型使用场景包括文本编辑器里的字符渲染、游戏中的子弹或NPC模板、数据库连接池（虽然更偏向池化，但思想有共通）。如果对象创建开销大且存在大量可共享数据，就可以考虑享元。

不过要注意线程安全，共享对象被多线程访问时得保证不可变性，否则容易出问题。状态一旦变了，缓存就不可靠了。



什么是依赖倒置原则？

依赖倒置原则讲的是高层模块不该依赖低层模块，它们俩都应该依赖抽象。抽象也不该依赖细节，反过来细节要依赖抽象。

举个实际场景，比如你写一个订单服务，里面要发邮件。如果直接在 OrderService 里 new EmailSender，那这个高层业务就被底层的邮件实现绑死了。想换成短信或者站内信？改代码都搞不定，得重编译。

正确的做法是定义一个 NotificationService 接口，OrderService 只依赖这个接口。具体是邮件还是短信，交给外部去实现和注入。这样换通知方式的时候，业务逻辑完全不用动。

Spring 框架就是靠这一套玩起来的。你用 @Service、@Autowired，本质上是在基于接口编程，运行时才把具体的 bean 注进来。IOC 容器干的就是解耦的事。

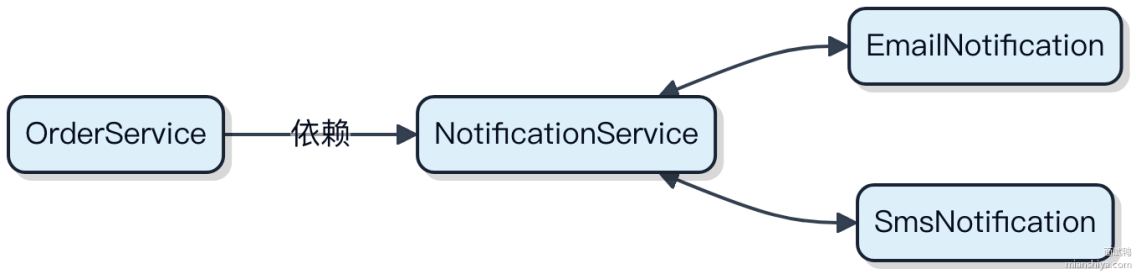
代码上大概是这样：

```
interface NotificationService {
    void send(String msg);
}

@Service
class EmailNotification implements NotificationService {
    public void send(String msg) { /* 邮件逻辑 */ }
```

```
}

@Service
class OrderService {
    private final NotificationService notifier;

    public OrderService(NotificationService notifier) {
        this.notifier = notifier;
    }
}
```

这样一来，OrderService 压根不经过具体实现，只跟接口打交道。新增渠道只需要加个新类实现接口，老代码零修改，开闭原则也满足了。

1）高层模块不能依赖低层模块
2）抽象不应依赖细节
3）细节必须依赖抽象



什么是代理模式？一般用在什么场景？

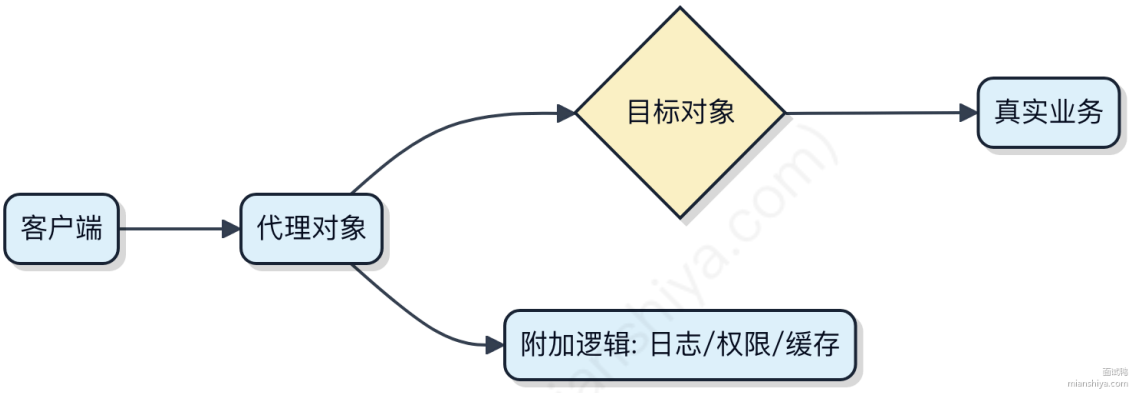
代理模式本质是为对象提供一个**代理**，用来控制对原对象的访问。你调用的是代理类，它帮你把请求转发给目标对象，中间可以加逻辑。

1）最常见的就是 Spring 的 AOP，比如你加个 `@Transactional` 注解，Spring 就会通过代理在方法前后开启和提交事务。实际调用的不是你的原始 bean，而是生成的代理对象。

2）远程调用也能用代理，像 Dubbo 里的接口注入，看着是本地调用，其实是代理帮你做了网络通信、序列化这些脏活累活。

3）还有懒加载，比如 Hibernate 关联对象没查出来前是个代理，真正 get 时才去数据库捞数据，避免一次性加载太多。

代码上分静态代理和动态代理。静态代理手写一个类实现相同接口，动态代理可以用 JDK 原生的 `Proxy` + `InvocationHandler`，或者 CGLIB 字节码增强。



注意别滥用。如果只是简单转发没意义，反而增加复杂度。重点是“在不修改目标的前提下增强行为”，这是它的价值所在。像日志记录、性能监控、限流降级，都是典型场景。

工厂模式和抽象工厂模式有什么区别？

工厂模式和抽象工厂模式都用来解耦对象创建，但解决的问题层次不一样。

工厂模式针对的是**单个产品等级结构**。比如你要生产不同类型的数据库连接，MySQLConnection、PostgreSQLConnection 都实现 Connection 接口。这时候用一个 DatabaseConnectionFactory，根据传入的类型返回具体实例。核心是把 new 操作统一收口，后续加新数据库类型也不用改业务代码。

```
public class ConnectionFactory {
    public Connection getConnection(String type) {
        if ("mysql".equals(type)) return new MySQLConnection();
        if ("pg".equals(type)) return new PostgreSQLConnection();
        throw new IllegalArgumentException();
    }
}
```

抽象工厂模式处理的是**多个产品族的创建**，强调产品之间的协调性。比如你有 MySQL + Redis 组合、PostgreSQL + Memcached 组合，每个组合是一套数据访问层方案。抽象工厂定义一套创建接口，每种实现负责创建一整套配套的对象。

```
interface DaoFactory {
    Connection createConnection();
    Cache createCache();
}

class MysqlRedisFactory implements DaoFactory {
    public Connection createConnection() { return new MySQLConnection(); }
    public Cache createCache() { return new RedisCache(); }
}
```

1）工厂模式关注单一类型对象的不同实现
2）抽象工厂关注一组相关或依赖对象的批量创建
3）抽象工厂通常会用到多个内部工厂或工厂方法来构建完整产品族

像 Spring 的 `BeanFactory` 是典型的工厂模式，而 Java EE 中的数据源与事务管理器协同场景就适合抽象工厂。一般项目里工厂模式更常见，抽象工厂容易过度设计，除非你真有多套独立且完整的组件组合要维护。

谈谈你了解的最常见的几种设计模式，说说他们的应用场景

工厂模式解决对象创建的复杂性问题。当系统需要根据条件生成不同类型的对象时，比如解析不同格式文件（XML、JSON），用工厂就能把创建逻辑集中管理，调用方不用关心具体类名。

1）简单工厂：写一个静态方法，传个参数返回对应实例，适合产品种类少的情况
2）工厂方法：每个子类对应一个工厂类，符合开闭原则，新增类型不影响原有代码
3）抽象工厂：生产一整套相关的产品族，比如 MySQL 和 Oracle 的 DAO 实现分别由不同的工厂批量产出

单例模式保证一个类只有一个实例。常见的做法是私有构造器 + 静态变量 + 公共静态访问方法。注意线程安全问题，推荐使用**枚举**实现，既防反射攻击又能防止序列化破坏单例。

```
public enum Singleton {
    INSTANCE;
    public void doSomething() { /*...*/ }
}
```

观察者模式定义了一对多的依赖关系，主题变化时通知所有观察者自动更新。典型场景是事件驱动系统，比如 Spring 的 ApplicationEvent 机制，监听 ContextRefreshedEvent 做初始化动作。

代理模式在不改变原始类接口的前提下增强功能。静态代理手动写，动态代理可以用 JDK 原生（基于接口）或 CGLIB（基于继承）。Spring AOP 默认用 JDK 动态代理，除非目标类没有接口才退化到 CGLIB。

模板方法模式把算法骨架固定在父类，把可变步骤延迟到子类实现。像 Java 中的 AbstractExecutorService.submit() 就是模板，真正执行交给具体线程池策略。

这几个模式在框架里到处都是，MyBatis 用工厂造 SqlSession，Spring 大量使用代理和观察者，理解它们等于看懂了框架的筋骨。

单例模式有哪几种实现？如何保证线程安全？

单例模式的关键是确保一个类只有一个实例，并提供全局访问点。常见的实现方式有好几种，每种都有它的适用场景和坑。

1）饿汉式在类加载时就创建实例，写法最简单，天然线程安全，但可能浪费资源。

```
public class Singleton {
    private static final Singleton INSTANCE = new Singleton();
    private Singleton() {}
    public static Singleton getInstance() {
        return INSTANCE;
    }
}
```

2）懒汉式延迟初始化，但直接加 synchronized 方法性能差，毕竟锁的是整个方法。

3）双重检查锁定（DCL）是优化方案，只在实例未创建时才同步，注意必须用 volatile 修饰实例变量，防止指令重排序导致返回未初始化完成的对象。

4）静态内部类方式利用了类加载机制保证线程安全，又实现了懒加载。外部类加载时不触发内部类初始化，直到第一次调用 getInstance 才加载 Holder 类并创建实例。

```
public class Singleton {
    private Singleton() {}
    private static class Holder {
        static final Singleton INSTANCE = new Singleton();
    }
    public static Singleton getInstance() {
        return Holder.INSTANCE;
    }
}
```

5）枚举单例最简洁安全，不仅能防反射攻击，还能防止序列化破坏单例，Effective Java 推荐的方式。

```
public enum Singleton {
    INSTANCE;
}
```

一般项目里用静态内部类或枚举就够了。DCL 虽高效但容易写错，比如漏掉 volatile。如果涉及到反序列化或反射的场景，优先选枚举。

你认为好的代码应该是什么样的？

好的代码首先得让别人能看懂，其次才是让机器执行。你写的每一行代码，其实都是在写文档，只不过这个文档刚好能运行。

可读性永远排第一。变量命名别缩写成谜语，函数职责要单一，一个方法干一件事就足够了。比如 `calculateTax()` 就别顺手把日志也打了，更别去改全局状态。这类脏活累活得交给专门的地方处理。

1）逻辑清晰比技巧炫技重要得多。三目运算嵌三层？递归套异步？省省吧，维护的人会骂你。简单 if-else 虽然啰嗦，但谁都看得明白。

2）边界条件必须处理。空指针、网络超时、数组越界，这些不是“理论上不会发生”，而是“一定会发生”。好的代码会在一开始就考虑这些情况，像 `Optional` 处理可能为空的返回值，就是一种防御性编程。

3）可测试性是硬指标。如果一个类依赖了 5 个静态方法和数据库连接，根本没法写单元测试，那设计就有问题。依赖注入、接口抽象，这些东西不是为了炫概念，是为了让你能搞不定线上 bug 时，至少能在本地复现。

```
public Optional<User> findUser(int id) {
    return userRepository.findById(id); // 返回 Optional，调用方必须处理 null 情况
}
```

最后，代码风格要统一。团队用 Lombok 就都用，不用就都别上。格式化工具该上就上，别让换行和缩进引发代码评审战争。一致性本身就能降低认知成本。

23 种设计模式分为哪三大类？

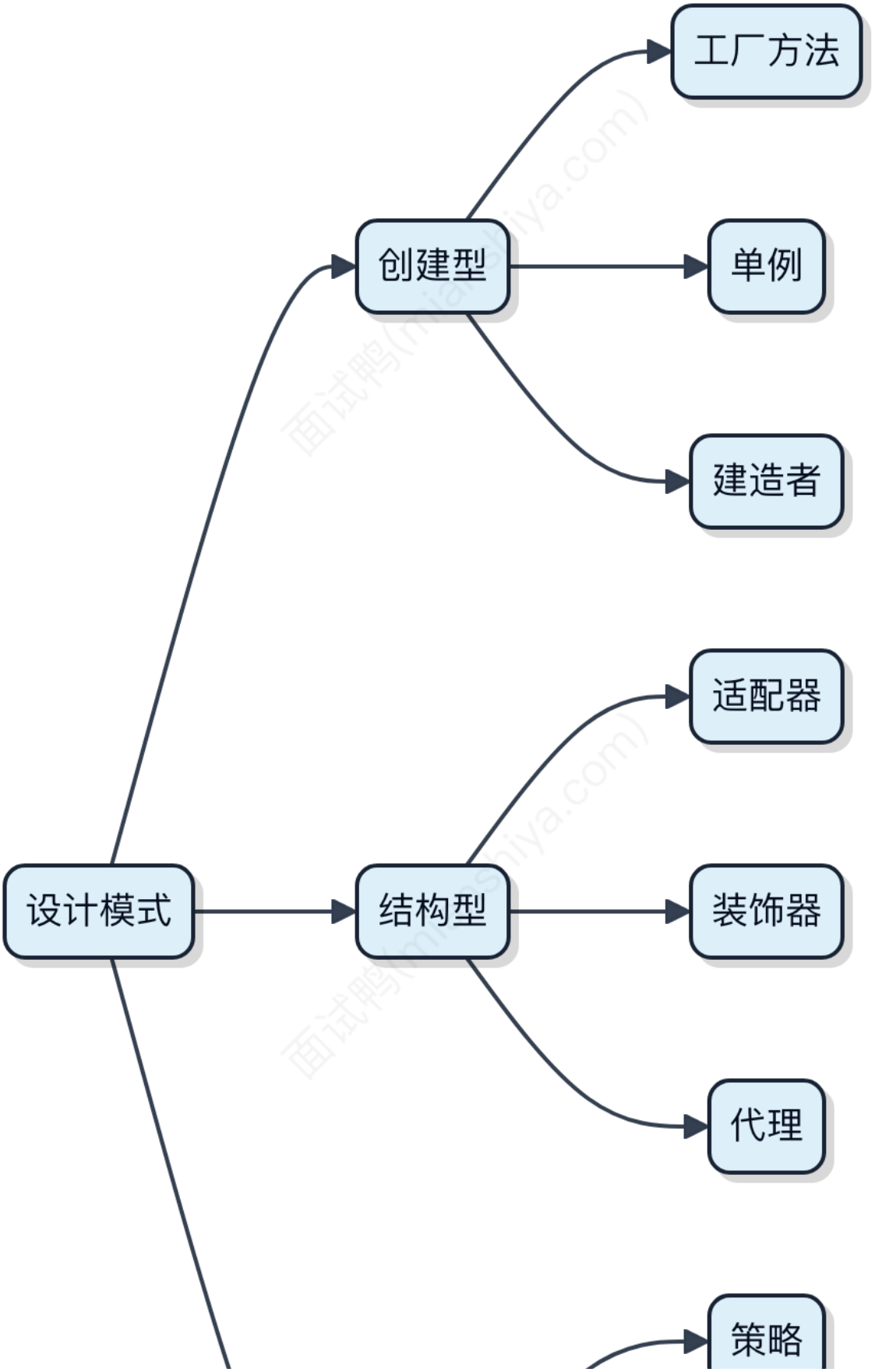
设计模式这东西，说白了就是前人踩坑总结出来的套路本子。23 种模式一般按解决的问题场景分成三类。

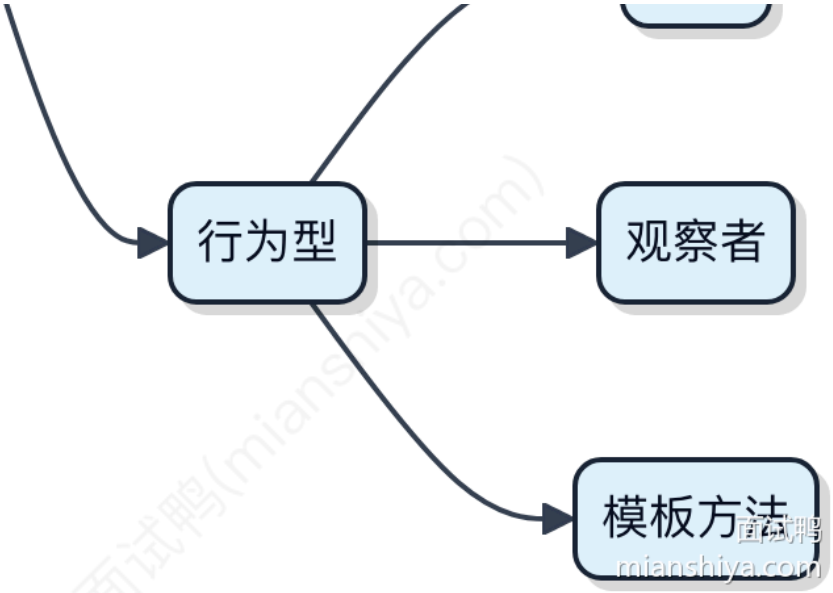
1）创建型模式
关注对象怎么生出来，把创建逻辑和使用逻辑拆开。比如你用 new 直接耦合了具体类，后面想换实现就麻烦。工厂方法、抽象工厂、建造者这些，都是为了让你**解耦创建过程**。Spring 里的 BeanFactory 就是典型的工厂思想。

2）结构型模式
重点在怎么组合类或对象形成更大的结构。比如你想给老系统加功能，但又不想改原来的代码。适配器模式就能让不兼容的接口能一起工作，装饰器模式可以在不修改原类的情况下动态加功能。Java I/O 流里的 `BufferedInputStream` 套 `FileInputStream` 就是装饰器的经典用法。

3）行为型模式
处理对象之间的职责分配和通信机制。比如一堆条件判断可以换成策略模式，状态切换复杂可以用状态模式。观察者模式在事件驱动场景里特别常见，像 Spring 的 ApplicationEvent 事件机制底层就用了它。

面试鸭(mianshiya.com)


设计模式
创建型
工厂方法
单例
建造者
结构型
适配器
装饰器
代理
策略



> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*